CORPORATE WORKSHOP PROPOSAL

Engineering an Agentic Harness with Crew AI

From autonomous agents to controlled crews, stateful flows and observable production systems

Detail	Information
Prepared for	AI companies and AI Engineer Who build Enterprise AI Product
Prepared by	Ashish Patel
Proposed date	17 October or 24 October 2026
Format	One-day, hands-on workshop | TBD
Participants	100 to 150 people
Duration	9:00 AM to 5:30 PM (adjustable)

1. Overview

Goal: teach your team to build AI agents that are controlled, observable and ready for production, using CrewAI.

What participants build: one system, step by step. It is an Enterprise AI Research & Decision Assistant that answers from your own knowledge, remembers context, works as a team of agents, asks for human approval before risky actions, and can be traced and tested.

Format: 100% hands-on. Every module is a short concept talk followed by a lab.

Audience: engineers, data scientists, architects and technical leads with basic Python. No prior agent experience is needed.

Outcome: a working harness, a reusable codebase and a production-readiness checklist.

> **Core idea**
> A strong AI model is like a strong horse. It still needs a harness (tools, knowledge, memory, workflow, control and verification) before you can trust it with real work. This workshop teaches how to build that harness.

2. Tailored to your company: your knowledge inside the agent

A key part of the day is grounding the agent in your own knowledge using RAG (retrieval-augmented generation):

- Your documents (for example policies, product documentation, FAQs or architecture notes) are split into chunks, converted into embeddings and stored in a vector database.
- The agent retrieves the relevant pieces and answers from them, with sources, instead of guessing.
- **Two options for the RAG lab:** (1) sample documents that we provide, which need no data sharing and are the safest default; or (2) your own non-sensitive documents, which are more relevant but need your approval and data policy first.
- Optional demonstration: a Planner agent that reads a knowledge folder and drafts a structured plan or proposal. It shows RAG, memory and guardrails working together.

3. What participants will be able to do

1. Build an agent with a clear role, goals and safety limits
2. Give agents tools and connect external tools through MCP
3. Ground answers in company documents (RAG)
4. Add scoped, persistent memory
5. Turn a team of agents into a controlled workflow with state, branching and capped retries
6. Add guardrails and human approval before risky actions
7. Trace, test and evaluate agent behaviour
8. Judge production readiness using a launch checklist

4. What we will teach: curriculum

Module	Topic	What we teach	Hands-on lab
Intro	Agent = model + harness	What an agent is, why a strong model still needs a harness, and the seven-layer map.	Discussion: what would you add before letting an unsafe agent into production?
M1	Agent loop and model choice	Roles, goals, limits, reasoning and planning, model routing and fallbacks.	Build the first agent with step, time and retry limits.
M2	Tools and MCP	Tool design, typed inputs, least privilege, MCP as a standard connector.	Build 3 custom tools and connect one MCP server.
M3	Context and knowledge (RAG)	Context vs knowledge vs memory, chunking, embeddings, retrieval, citing sources.	Query the knowledge base (sample or company documents) and answer with sources.
M4	Memory	Short and long-term memory, scopes, consolidation, privacy.	Add scoped agent memory and a read-only shared slice, then inspect the memory tree.
M5	Orchestration	One agent vs a crew, flow vs crew, branching, loops with caps.	Build a Researcher, Analyst and Critic crew inside a Flow with a check and retry loop.
M6	Control	Guardrails, human approval, sandboxing, prompt injection, governance.	Add input and output guardrails and an approval gate before an email is sent.
M7	Resilience	Retries, timeouts, checkpoints, safe repeats.	Demonstration: crash the process, then resume from a checkpoint.
M8	Verification	Tracing, evaluation, test sets, cost and latency budgets.	Trace a run and score it with tests.
Capstone	Fix the broken harness	Diagnosing failures by layer.	Teams find and fix 8 planted failures.
Close	Going to production	Lifecycle, maturity ladder, common mistakes, launch checklist.	Self-assess your own harness.

5. Event schedule

Proposed time: 9:00 AM to 5:30 PM. This can be adjusted to your working hours.

Time	Session
8:30 – 9:00	Arrival, registration and environment check
9:00 – 9:20	Welcome and introduction: agent = model + harness
9:20 – 10:05	M1: Agent loop and model choice
10:05 – 10:50	M2: Tools and MCP
10:50 – 11:05	Tea break
11:05 – 11:50	M3: Context and knowledge (RAG)
11:50 – 12:20	M4: Memory
12:20 – 1:15	Lunch
1:15 – 2:15	M5: Orchestration, from crew to flow
2:15 – 3:00	M6: Control, guardrails and human approval
3:00 – 3:15	Tea break
3:15 – 3:35	M7: Resilience, checkpoint and resume
3:35 – 4:15	M8: Verification, tracing and evaluation
4:15 – 5:00	Capstone: fix the broken harness
5:00 – 5:15	Going to production: lifecycle, maturity and checklist
5:15 – 5:30	Recap, Q&A and feedback

6. What we need from the company

Item	Detail
Participants	[N] people, ideally 30 or fewer for the best support.
Room	Tables for laptops, projector or screen, reliable wifi and power at each seat.
Laptops	Participants bring their own, with Python 3.10 or later installed.
Network	Access to the model API and package downloads. Please check firewall rules.
Model access	(Free) Ollama Preferred for Local(Minimal 8GB ram Required), or Any API keys is fine. Please share your data policy.
Knowledge documents	Optional: non-sensitive documents for the RAG lab, shared [1 week] before the event.
Catering	Tea and coffee, lunch and two breaks.
Pre-work	A short setup and smoke test, sent one week before the event.

7. What we provide

- Facilitator and [1 to 2] lab helpers
- Slide deck with real-world analogies and diagrams
- Workshop code repository with starter code and solutions for each lab
- Sample knowledge documents for the RAG lab
- Capstone exercise with an answer key
- Production-readiness checklist and a one-page recap
- Post-event support: slides, code and [Q&A window, if agreed]

8. Pre-work for participants

Sent one week before the event:

1. Install Python 3.10 or later and CrewAI (pinned versions provided)
2. Clone the workshop repository
3. Set up the API key and embedding provider
4. Run the smoke test and send us the result

9. Data and privacy

- Document text and memory content are sent to the configured embedding and language model providers. If your documents are confidential, we can use a company-approved provider or a local model.
- We recommend starting with sample documents and moving to company documents only with your approval.
- Nothing from the workshop needs to be stored on our side. You keep the code and the knowledge base.

10. Confirmation checklist

Please confirm the following so we can finalise the plan:

1. Date, venue or online, and headcount
2. Audience level and roles
3. Model access: company keys or shared keys
4. Whether the RAG lab uses sample or company documents, and any compliance limits
5. Whether you would prefer a shorter half-day version or a longer two-day version with deeper labs

Contact
Ashish Patel | ashishpatel.ce.2011@gmail.com | +91 8460238402